

Task 67: Accounts Merge

Problem Description

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account. Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name. After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Java Solution

```

class Solution {
    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        Map<String, String> emailToName = new HashMap<>();
        Map<String, String> parent = new HashMap<>();

        for (List<String> account : accounts) {
            String name = account.get(0);
            for (int i = 1; i < account.size(); i++) {
                String email = account.get(i);
                emailToName.put(email, name);
                parent.putIfAbsent(email, email);
                union(parent, account.get(1), email);
            }
        }

        Map<String, TreeSet<String>> merged = new HashMap<>();
        for (String email : emailToName.keySet()) {
            String root = find(parent, email);
            merged.computeIfAbsent(root, k -> new TreeSet<>()).add(email);
        }

        List<List<String>> res = new ArrayList<>();
        for (String root : merged.keySet()) {
            List<String> list = new ArrayList<>();
            list.add(emailToName.get(root));
            list.addAll(merged.get(root));
            res.add(list);
        }
        return res;
    }

    private String find(Map<String, String> parent, String s) {
        if (!parent.get(s).equals(s)) {
            parent.put(s, find(parent, parent.get(s)));
        }
        return parent.get(s);
    }

    private void union(Map<String, String> parent, String s1, String s2) {
        String root1 = find(parent, s1);
        String root2 = find(parent, s2);
        if (!root1.equals(root2)) {
            parent.put(root1, root2);
        }
    }
}

```

	}
}	